Model-Driven DSL for Predictive Maintenance of Servo Motors

Ismat Abou Khachfe
TU Ilmenau
Model-Driven Software Engineering
Prof. Ralph Maschotta

March 2026

Contents

1 Project Description **2**

2 Metamodel **2**

3 Editor **4**

4 Model to text transformation Template **5**

5 Application Examples **5**
5.1 Moderate Demo Model 5

6 Conclusion **6**

1 Project Description

The Project is to develop a Domain-Specific Language (DSL) for predictive maintenance of small machines, specifically targeting servo motors monitored via sensors connected to an Arduino-like microcontroller board. Predictive maintenance involves continuous sensor data acquisition, anomaly detection via configurable rules, and automated alerts via actuators to prevent failures.

Sensors supported include DHT11 (temperature/humidity), DigitalTemperature, VibrationBigSound, and VibrationSmallSound, with units like °C or analog levels. Actuators ("actors") include RGB LEDs (Green/RED/Blue for status), ActiveBuzzer, and PassiveBuzzer for warnings. The DSL separates hardware wiring (structure) from monitoring logic (behavior), enabling non-experts to model and generate deployable Arduino code.

The project uses Eclipse Modeling Framework (EMF) for the metamodel, Sirius for dual-view editors (structure and behavior diagrams), and Acceleo for model-to-text (M2T) code generation to Arduino `.ino` sketches.

2 Metamodel

The root element of the Metamodel is MaintenanceSysten class which contains SystemStructure and SystemBehavior.

Key Classes

- `Machine (Structure)`
  - Attributes: `name`, `model` (e.g., "Servo"), `manufacturer`
  - Contains: `Sensors`
- `Sensor (Structure)`
  - Attributes: `name`, `type` (enum: `DHT11`, `DigitalTemperature`, `VibrationBigSound`, `VibrationSmallSound`), `unit`
  - Associated with: `PinGroups`
- `Board (Structure)`
  - Contains: `Machine`, `PinGroups`
  - Pins defined by `PinType`: `DIGITAL_IN`, `DIGITAL_OUT`, `ANALOG_IN`, `ANALOG_OUT` ,`GND` , `VCC`
  - PinGroup class contains Pin class
- `Connection (Structure)`
  - Links: source `Pin` → target `Pin`
- `MonitoringRule (Behavior)`
  - Attributes: `severity` (Normal / Warning / Critical), `enabled`, `AlertColor`
  - Observes: `Sensors`
  - Has `Condition`:
    - `RangeCondition`
    - `ThresholdCondition` (with `Operator enumeration`)
    - `RateOfChangeCondition`

  - Triggers: `Alert` (`ActionType`: `RGB_Led`, `ActiveBuzzer` , `PassiveBuzzer`)

- `Enumerations`
  - `AlertColor`: `Green`, `Red`, `Blue`, `NaN`
- `Why Alert is a hardware (SystemStructure) ? and why is it separated from the other components?`
  - *Architectural rationale:* Alert is modeled as a physical actuator (e.g., LED, buzzer, relay) within the `SystemStructure` because it requires explicit pin configuration and wiring, making it comparable to sensors as output hardware.
  - This follows the *Separation of Concerns* principle by distinguishing:
    - The **structural layer** (physical devices and wiring)
    - The **behavioral layer** (monitoring logic and rules)
  - Separating `Alert` from `MonitoringRule` improves reusability, allowing multiple rules to reference the same actuator without duplicating hardware configuration.
  - This ensures low coupling between infrastructure and logic, enabling independent evolution of hardware topology and monitoring behavior.
  - It also mirrors real-world IoT workflows, where physical deployment (wiring sensors/actuators) is performed independently from behavioral configuration.

Design Overview This design captures both the hardware topology and rule-based anomaly detection, enabling automated code generation for real-time monitoring.

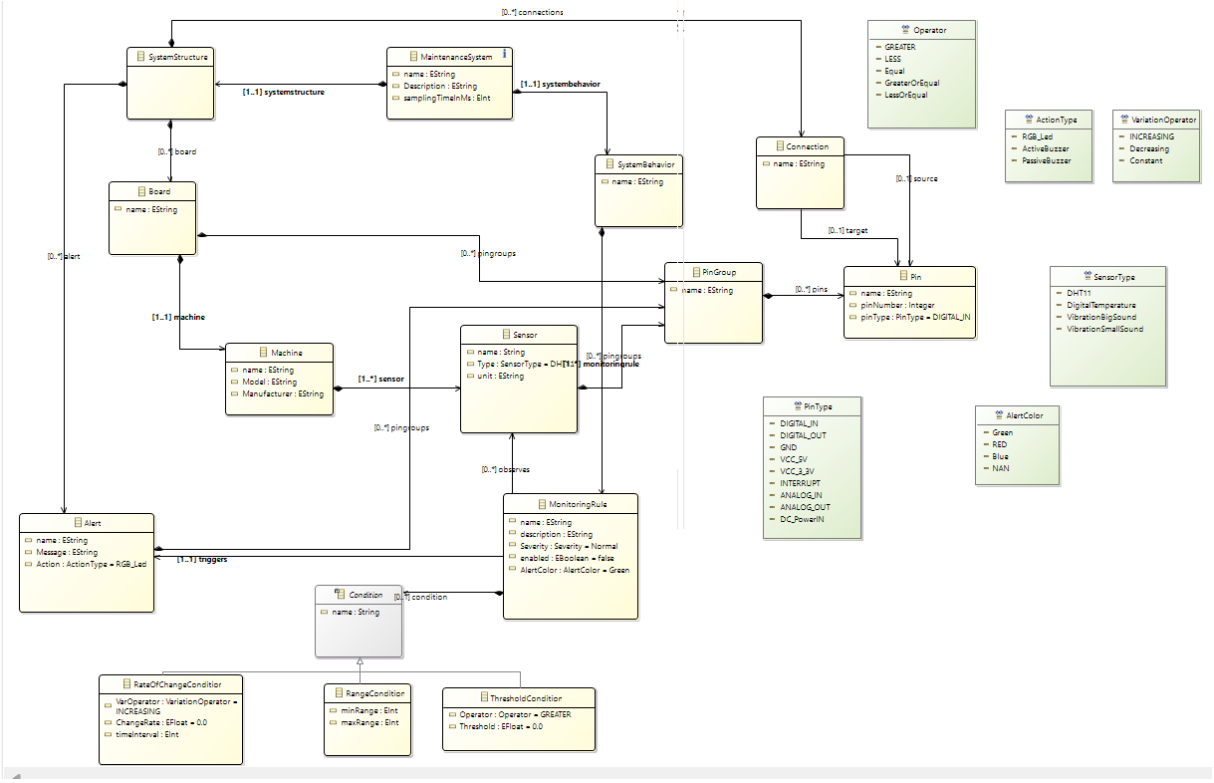


Figure 1: Metamodel

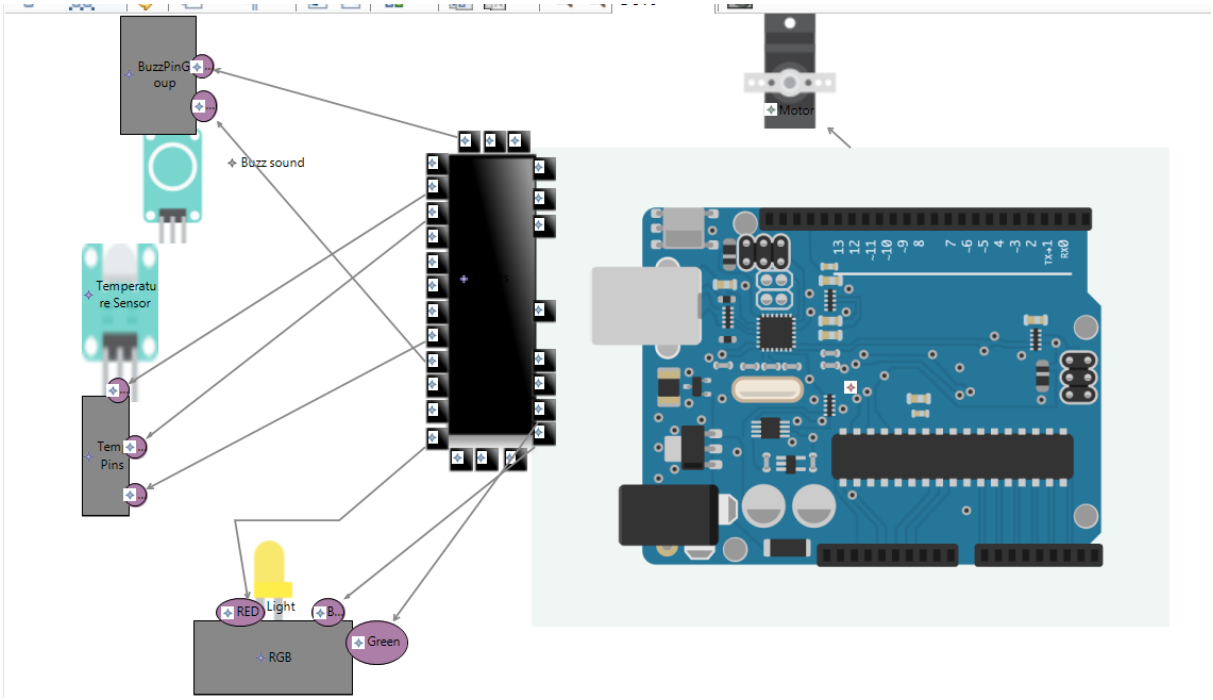


Figure 2: System Structure of the demo model

3 Editor

The model editor is implemented with **Eclipse Sirius**, providing two diagram representations for usability:

- **System Structure Diagram**
- **System Behavior Diagram**

This separation reduces complexity—users first plan hardware, then add behavior:

- **Hardware Planning**
  - Nodes: `BoardNode`, `Machine`, `SensorNode`, `AlertNode`
  - Tools: `Section Structure Creation Tools`
  - The references between components are represented visually by arrows (Relation Edge)
- **Behavior Definition**
  - Nodes: `MonitoringRule`, `SensorNode`, `Observes`, `Triggers`, `AlertNode`
  - Tools: `Section Behavior Tools (Creation Tools)`
  - The references between components and rules are represented visually by arrows (Relation Edge)

Example Diagrams

Diagram 2 is an example of the diagram of the structure of the model.

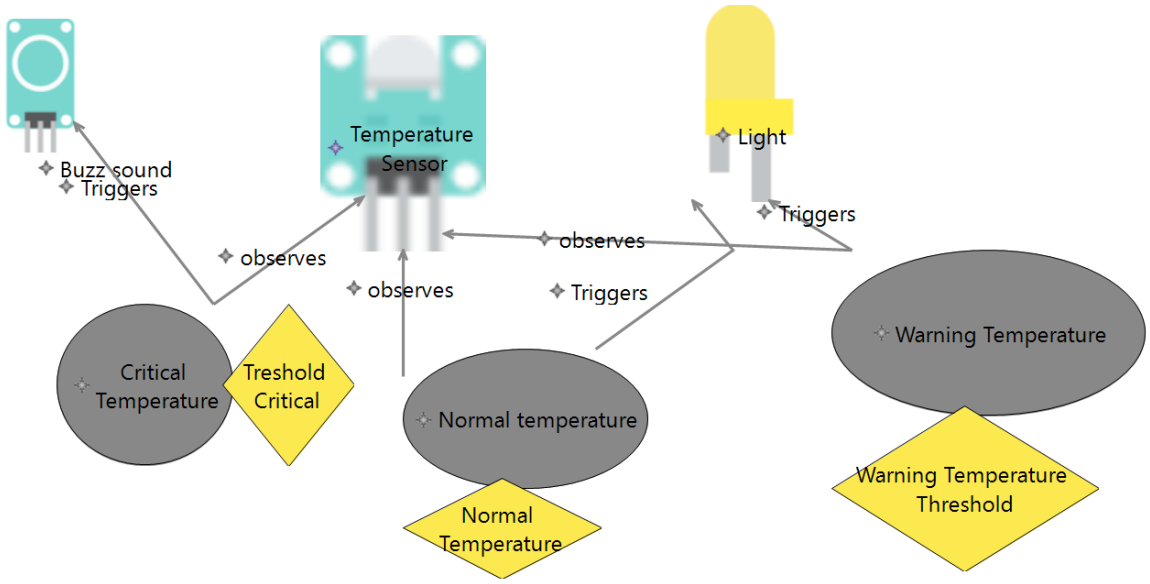


Figure 3: System Behavior diagram of the demo

Diagram 3 is an example of the behavior.

Validation and Tooling Validation rules include:

- `RangeCoherence` (ensures `minRange < maxRange`)
- `MonitoringRulecondition` (checks if condition is null)

Users create nodes via palettes, draw connections, and get live feedback on structure, pin compatibility and rule consistency.

4 Model to text transformation Template

The Acceleo generator (`generate.mtl`) performs model-to-text transformation, producing deployable Arduino `.ino` code from any valid `MaintenanceSystem` instance. It traverses structure for pin setup/includes (e.g., DHT.h for DHT11), generates sensor read functions (analogRead-/digitalRead), monitoring checks (if-conditions matching rules), and alert outputs (digitalWrite/-tone).

Key mappings: - Sensors → read functions (e.g., DHT11 reads temperature). - MonitoringRules → check functions (e.g., if temperature $>$ 30.0 → RED LED). - Pins/Connections → pinMode, wiring. - Alerts → HIGH/LOW or tone.

Generated code includes servo simulation, debug prints, and non-blocking loop with user-defined sampling. It handles polymorphism (Range/Threshold conditions) via OCL queries.

5 Application Examples

5.1 Moderate Demo Model

The demo models a servo motor with one DHT11 temperature sensor. Structure connects sensor/alert pins. Behavior has three rules: - Normal: 15–30°C → Green LED. - Warning: >30°C

→ RED LED. - Critical: >31°C → ActiveBuzzer.

Generation yields `Demo_For_MDSE.ino` with DHT reads, threshold checks, pin I/O—ready for Arduino upload. Scenario: overheating triggers buzzer/LEDs while servo sweeps, urging the user to stop the motor or place it in a better condition.

First, the temperature of the motor is between 15 and 30 °C, so the functioning is normal, and we get a green LED. Then, after a few minutes, the temperature rises above 30 °C, signaling a warning. After some time with the motor running at full speed, it becomes critical and the temperature rises above 31 °C, which causes not only a red light but also a very loud buzzing sound to urge the user to take the necessary measures to prevent the motor from overheating.

6 Conclusion

This project presents a minimal Domain-Specific Language (DSL) for predictive maintenance of servo motors using Arduino-based sensor monitoring. The goal was not to build a fully industrial solution, but rather to design a simple and clear DSL that enables a person with basic knowledge of maintenance and Arduino sensors to generate a functional monitoring program and detect abnormal changes in a machine's health.

Although the DSL is intentionally lightweight, it can be extended to support more realistic and complex maintenance scenarios. The structure–behavior separation, validation mechanisms, and automatic code generation demonstrate how model-driven engineering can simplify the development of embedded monitoring systems.

Regarding behavior modeling, a rule-based approach was implemented for clarity and simplicity. More advanced methods studied during the course, such as using opaque behaviors or other behavioral modeling techniques, could have been applied to achieve a more sophisticated solution. Nevertheless, the chosen approach remains effective, understandable, and fully operational for the intended purpose.

Finally, I would like to thank Prof. Maschotta for the opportunity to work on this project and explore the practical application of model-driven software engineering in the context of predictive maintenance.